8강. 비동기 처리(fetch/axios)

javascript 비동기 처리

- **비동기 처리**

  자바스크립트는 싱글 스레드 방식으로 처리해야 할 기능을 순차적으로 처리함. 비동기 처리는 프로그램에서 처리해야 할 기능들을 순차적으로 처리하는 것이 아니라, 원하는 순서에 맞게 처리하는 것

방식	버전	기능
콜백 함수	기존부터 사용	함수 안에 또 다른 함수를 매개변수로 넘겨서 실행 순서를 제어함
프로미스(Promise)	에크마스크립트 2015	프로미스 객체와 콜백 함수를 사용해서 실행 순서를 제어함
async/await	에크마스크립트 2017	async와 await를 사용해서 실행순서 제어

javascript 비동기 처리

- **동기 처리 – async01.js**

```javascript
function displayA() {
  console.log('A');
}

function displayB() {
  // console.log('B');
  // 2초 후에 'B'를 출력하는 비동기 처리
  setTimeout(() => {
    console.log('B');
  }, 2000);
}

function displayC() {
  console.log('C');
}

displayA();
displayB();
displayC();
```

```javascript
/*싱글 스레드이므로 동기적으로 실행됨
  displayA() -> displayB() -> displayC() 순서로 실행됨
  displayB()는 2초 후에 'B'를 출력하는 비동기 처리이므로,
  displayC()가 먼저 실행되어 'C'가 출력됨.
  'A' -> 'C' -> 'B' 순서로 출력됨*/
```

```
A
C
B
```

javascript 비동기 처리

- **비동기 처리 – async02.js**

```javascript
function displayA() {
  console.log('A');
}

function displayB(callBack) {
  setTimeout(() => {
    console.log('B');
    callBack();
  }, 2000);
}

function displayC() {
  console.log('C');
}

displayA();
displayB(displayC);
```

```javascript
/*
  비동기 처리 - 프로그램에서 처리해야 할 기능들을 순차적으로
  처리하는 것이 아니라, 동시에 처리하는 방식
  원하는 순서에 맞게 처리하기 위해 콜백함수를 사용
  'A' -> 'B' -> 'C' 순서로 출력됨
*/
```

```
A
B
C
```

javascript 비동기 처리

- **비동기 처리 – async03.js**

```javascript
async function displayA() {
  console.log('A');
}

async function displayB() {
  // Promise 객체를 반환하여 비동기 작업 처리
  return new Promise((resolve) => {
    setTimeout(() => {
      console.log('B');
      resolve();
    }, 2000);
  });
}

async function displayC() {
  console.log('C');
}
```

```javascript
/*
  비동기 처리 - async/await 방식
  async/await를 사용하여 더 깔끔하게 비동기 작업 처리
  Promise를 명시적으로 사용하지 않음
  'A' -> 'B' -> 'C' 순서로 출력됨
*/
```

javascript 비동기 처리

- **비동기 처리 – async03.js**

```
// async/await를 사용하여 순차적으로 실행
async function runSequence() {
  await displayA();
  await displayB();
  await displayC();
}

runSequence(); // runSequence() 함수를 호출하여 실행
```

```
A
B
C
```

리엑트에서 비동기 처리

- **리엑트에서 비동기 처리**

  작업이 끝날때 까지 기다리지 않고 다음 코드를 먼저 실행하는 방식

  예)
  - 서버에서 데이터 가져오기 (API 호출)
  - 파일 업로드
  - 로그인 요청

- **기본 동작 흐름**

1. 컴포넌트 렌더링
2. useEffect 실행
3. API 요청 (fetch / axios)
4. 데이터 받아오기
5. useState로 저장
6. 화면 다시 렌더링

리엑트에서 비동기 처리

- **useEffect()를 사용하는 이유**

```
useEffect(() => {
  // API 호출
}, [ ]);
```

• 컴포넌트가 처음 렌더링될 때 실행됨
• 무한 호출 방지
"화면 뜨자마자 데이터 가져오기" 역할

- **상태와 비동기의 관계**

```
const [data, setData] = useState([]);
```

서버에서 받은 데이터를 저장하면:
• setData 실행
• 컴포넌트 자동 재렌더링
• 화면 업데이트

리엑트에서 비동기 처리

- **fetch() 와 axios() 차이**

항목	fetch	axios
기본 제공 여부	브라우저 내장 API	외부 라이브러리 (설치 필요)
설치	❌ 필요 없음	✅ `npm install axios`
데이터 변환	`response.json()` 직접 호출	자동 JSON 변환
응답 데이터 접근	`res.json()`	`res.data`
에러 처리	HTTP 에러 자동 처리 ❌	HTTP 에러 자동 throw ✅

리엑트에서 비동기 처리

- **json Placeholder**

JSONPlaceholder는 가짜 데이터가 필요할 때 언제든지 사용할 수 있는 무료 온라인 REST API입니다

{JSON} Placeholder

Free fake and reliable API for testing and prototypi

Powered by JSON Server + LowDB.

Serving ~3 billion requests each month.

```
fetch('https://jsonplaceholder.typicode.com/todos/1')
      .then(response => response.json())
      .then(json => console.log(json))
```

스크립트 실행

```
{
  "userId": 1,
  "id": 1,
  "title": "delectus aut autem",
  "completed": false
}
```

리엑트에서 비동기 처리

- **json 데이터**


```
[
  {
    "userId": 1,
    "id": 1,
    "title": "delectus aut autem",
    "completed": false
  },
  {
    "userId": 1,
    "id": 2,
    "title": "quis ut nam facilis et officia qui",
    "completed": false
  },
  {
    "userId": 1,
    "id": 3,
    "title": "fugiat veniam minus",
    "completed": false
  },
  {
    "userId": 1,
    "id": 4,
    "title": "et porro tempora",
    "completed": true
  },
```

fetch()를 사용한 비동기 처리

- fetch()를 사용한 비동기 처리

할 일(To-do) 데이터

- delectus aut autem
- quis ut nam facilis et officia qui
- fugiat veniam minus
- et porro tempora
- laboriosam mollitia et enim quasi adipisci quia provident illum
- qui ullam ratione quibusdam voluptatem quia omnis
- illo expedita consequatur quia in
- quo adipisci enim quam ut ab
- molestiae perspiciatis ipsa
- illo est ratione doloremque quia maiores aut

fetch()를 사용한 비동기 처리

- fetch()를 사용한 비동기 처리 - FetchExample.jsx

```
const FetchExample = () => {
  const [data, setData] = useState([]);

  // 컴포넌트가 마운트될 때 한 번만 실행
  useEffect(() => {
    fetch("https://jsonplaceholder.typicode.com/todos")
      .then((response) => response.json()
      ) // JSON 응답을 JavaScript 객체로 변환
      .then((result) => {
          setData(result);
          console.log(result);
        }
      ) // 변환된 데이터를 상태에 저장
      .catch((error) => console.error(error)); // 오류 처리
  }, []);
```

fetch()를 사용한 비동기 처리

- fetch()를 사용한 비동기 처리

```
  return (
    <div>
      <h2>할 일(To-do) 데이터</h2>
      <ul>
        {data.map((todo) => (
          <li key={todo.id}>{todo.title}</li>
        ))}
      </ul>
    </div>
  );
};

export default FetchExample;
```

fetch()를 사용한 비동기 처리

- fetch()를 사용한 비동기 처리

할 일(To-do) 데이터

제목: delectus aut autem

완료 여부: ◯ 미완료

fetch()를 사용한 비동기 처리

- fetch()를 사용한 비동기 처리 - App.jsx

```
import FetchExample from './fetch_ex/FetchExample'
import FetchExample2 from './fetch_ex/FetchExample2'

function App() {

  return (
    <>
      {/* <FetchExample /> */}
      <FetchExample2 id={1} />
    </>
  )
}

export default App
```

fetch()를 사용한 비동기 처리

- fetch()를 사용한 비동기 처리 -FetchExample2.jsx

```
const FetchExample2 = ({ id }) => {
  // 객체 초기값을 null로 설정 - 데이터가 아직 로드되지 않았음
  const [data, setData] = useState(null);

  // JSONPlaceholder에서 사용자 데이터를 가져오는 예시
  useEffect(() => {
    fetch(`https://jsonplaceholder.typicode.com/todos/${id}`)
      .then((response) => response.json()
      ) // JSON 응답을 JavaScript 객체로 변환
      .then((result) => {
          setData(result);
          console.log(result);
        }
      ) // 변환된 데이터를 상태에 저장
      .catch((error) => console.error(error)); // 오류 처리
  }, [id]); // id가 변경될 때마다 데이터를 다시 가져옴
```

fetch()를 사용한 비동기 처리

- fetch()를 사용한 비동기 처리

```
  return (
    <div>
      <h2>할 일(To-do) 데이터</h2>
      {data && (
        <div>
          <p>
            <strong>제목:</strong> {data.title}
          </p>
          <p>
            <strong>완료 여부:</strong>
            {data.completed ? "● 완료" : "○ 미완료"}
          </p>
        </div>
      )}
    </div>
  );
};

export default FetchExample2;
```

json(javascript object notation)

- **Json이란?**

  JSON (JavaScript Object Notation) 은

  → 데이터를 **문자열 형태로 표현하는 규칙(포맷)** 입니다.

  - **기본 구조**

    JSON은 **키(key)와 값(value)** 으로 이루어져 있습니다.

```
{
  "name":  " 이순신 " ,
   " age " : 45,
  "isStudent": false
}
```

"name" → key (항상 문자열)

"홍길동" → value

json(javascript object notation)

- **배열**

```
{
  "fruits": ["사과", "바나나", "포도"]
}
```

- **객체**

```
{
  “product”: {
    “name”: “마우스",
    “price": 30000
  }
}
```

- **Json 사용**
  - 서버 ↔ 클라이언트 데이터 통신
  - API 응답 데이터 형식
  - 설정 파일 (config)
  - 데이터 저장 (localStorage 등)

json(javascript object notation)

- 로컬 JSON 파일과 통신하기

public/json/users.json

```
[
  {
    "id": 1,
    "name": "Leanne Graham",
    "username": "Bret",
    "email": "Sincere@april.biz",
    "address": {
      "street": "Kulas Light",
      "suite": "Apt. 556",
      "city": "Gwenborough",
      "zipcode": "92998-3874",
      "geo": {
        "lat": "-37.3159",
        "lng": "81.1496"
      }
    },
    "phone": "1-770-736-8031 x56442",
    "website": "hildegard.org",
    "company": {
      "name": "Romaguera-Crona",
      "catchPhrase": "Multi-layered client-server neural-net",
```

json(javascript object notation)

- 로컬 JSON 파일과 통신하기

```
  {
    "id": 3,
    "name": "Clementine Bauch",
    "username": "Samantha",
    "email": "Nathan@yesenia.net",
    "address": {
      "street": "Douglas Extension",
      "suite": "Suite 847",
      "city": "McKenziehaven",
      "zipcode": "59590-4157",
      "geo": {
        "lat": "-68.6102",
        "lng": "-47.0653"
      }
    },
    "phone": "1-463-123-4447",
    "website": "ramiro.info",
    "company": {
      "name": "Romaguera-Jacobson",
      "catchPhrase": "Face to face bifurcated interface",
      "bs": "e-enable strategic applications"
    }
  }
]
```

json(javascript object notation)

- 로컬 JSON 파일과 통신하기

회원 목록

ID	이름	도시	이메일
1	Leanne Graham	Gwenborough	Sincere@april.biz
2	Ervin Howell	Wisokyburgh	Shanna@melissa.tv
3	Clementine Bauch	McKenziehaven	Nathan@yesenia.net

json(javascript object notation)

- 로컬 JSON 파일과 통신하기

```
function FetchGet() {
  console.log("1. 컴포넌트 실행");
  const [users, setUsers] = useState([]);

  //useEffect는 데이터를 가져오는데 시간이 소요될때 사용
  useEffect(() => {
    console.log("2. useEffect 실행");

    fetch("/json/users.json")
      .then(response => response.json())
      .then(data => {
        console.log("3. 데이터 가져오기 성공", data);
        setUsers(data);
      })
      .catch(error => console.error("Error:", error));
  }, []);
```

json(javascript object notation)

- 로컬 JSON 파일과 통신하기

```
  return (
    <div className="container">
      <h2>회원 목록</h2>
      <ul>
        {users.map((user) => (
          <li key={user.id}>
            이름: {user.name}, 주소: {user.address.city}, 이메일: {user.email}
          </li>
        ))}
      </ul>
    </div>
  );
}
```

json(javascript object notation)

- 로컬 JSON 파일과 통신하기

사용자 정보 (ID: 2)

ID: 2
이름: Ervin Howell
도시: Wisokyburgh
이메일: Shanna@melissa.tv

json(javascript object notation)

- 로컬 JSON 파일과 통신하기

```
function FetchGetById({ id }) {
  const [user, setUser] = useState(null);

  useEffect(() => {
    fetch("/json/users.json")
      .then(response => response.json())
      .then(data => {
        const foundUser = data.find(user => user.id === id);
        setUser(foundUser);
      })
      .catch(error => console.error("Error:", error));
  }, [id]);
```

json(javascript object notation)

- 로컬 JSON 파일과 통신하기

```
  return (
    <div className="container">
      <h2>사용자 정보 (ID: {id})</h2>
      {user && (
        <div>
          <p><strong>ID:</strong> {user.id}</p>
          <p><strong>이름:</strong> {user.name}</p>
          <p><strong>도시:</strong> {user.address.city}</p>
          <p><strong>이메일:</strong> {user.email}</p>
        </div>
      )}
    </div>
  );
}

export default FetchGetById;
```

axios()를 사용한 비동기 처리

- **axios란?**

  HTTP 요청을 쉽게 보내기 위한 라이브러리

  - 서버(API)와 통신할 때 사용
  - GET, POST, PUT, DELETE 요청 처리
  - **fetch의 불편한 점** (JSON 변환 직접 해야 함,에러 처리 불편) **보완**

- **axios 라이브러리 설치**

```
npm install axios
```

```
"dependencies": {
  "axios": "^1.14.0",
  "react": "^19.2.4",
  "react-dom": "^19.2.4"
},
```

axios()를 사용한 비동기 처리

- **axios 메서드**

메서드	용도	axios 함수
GET	데이터 조회	axios.get()
POST	데이터 생성	axios.post()
PUT	전체 수정	axios.put()
PATCH	일부 수정	axios.patch()
DELETE	데이터 삭제	axios.delete()

axios()를 사용한 비동기 처리

- **REST API**
  - RESTful API는 웹에서 서버와 클라이언트가 데이터를 주고받는 방식을 설계하는 아키텍처 프레임이다.
  - REST (Representational State Transfer) 는 2000년 로이 필딩(Roy Fielding) 이 논문에서 제안한 웹 기반의 설계 원칙으로
  - "**리소스(Resource)를 URL로 표현하고, 그 리소스에 대한 행위는 HTTP 메서드로 구분한다**" 개념이다.
  - 행위는 **HTTP 메서드(GET, POST, PUT, DELETE 등)** 로 구분한다.

axios()를 사용한 비동기 처리

- axios()를 사용한 비동기 처리

할 일(To-do) 데이터

- delectus aut autem
- quis ut nam facilis et officia qui
- fugiat veniam minus
- et porro tempora
- laboriosam mollitia et enim quasi adipisci quia provident illum
- qui ullam ratione quibusdam voluptatem quia omnis
- illo expedita consequatur quia in
- quo adipisci enim quam ut ab
- molestiae perspiciatis ipsa
- illo est ratione doloremque quia maiores aut

axios()를 사용한 비동기 처리

- axios()를 사용한 비동기 처리 - AxiosGet.jsx

```
import React, { useState, useEffect } from "react";
import axios from "axios";

// JSONPlaceholder에서 사용자 데이터를 가져오는 예시
const AxiosGet = () => {
    const [data, setData] = useState([]);

    useEffect(() => {
        // JSONPlaceholder API에서 할 일 데이터를 가져옵니다.
        axios.get("https://jsonplaceholder.typicode.com/todos")
            .then((response) => {
                setData(response.data); // 응답 데이터를 response.data에 저장
                console.log(response.data);
            })
            .catch((error) => console.error(error)); // 오류 처리
    }, []);
```

axios()를 사용한 비동기 처리

- axios()를 사용한 비동기 처리

```
    return (
        <div>
          <h2>할 일(To-do) 데이터</h2>
          <ul>
            {data.map((todo) => (
              <li key={todo.id}>{todo.title}</li>
            ))}
          </ul>
        </div>
    );
}
```

axios()를 사용한 비동기 처리

- axios()를 사용한 비동기 처리 – AxiosGetById.jsx

```
import { useEffect, useState } from "react";
import axios from "axios";

const AxiosGetById = ({ id }) => {
  const [data, setData] = useState(null);

  useEffect(() => {
    // JSONPlaceholder API에서 특정 ID의 할 일 데이터를 가져옵니다.
    axios.get(`https://jsonplaceholder.typicode.com/todos/${id}`)
      .then((response) => {
        setData(response.data);
        console.log(response.data);
      })
      .catch((error) => console.error(error));
  }, [id]);
```

axios()를 사용한 비동기 처리

- axios()를 사용한 비동기 처리

```
return (
  <div>
    <h2>할 일(To-do) 데이터</h2>
    {data && (
      <div>
        <p>
          <strong>제목: </strong> {data.title}
        </p>
        <p>
          <strong>완료 여부: </strong>
          {data.completed ? "● 완료" : "○ 미완료"}
        </p>
      </div>
    )}
  </div>
);
}
```

axios()를 사용한 비동기 처리

- axios()를 사용한 비동기 처리

```
생성된 데이터:
▼ {title: '땀나게 운동하기', completed: false, id: 201}
    completed: false
    id: 201
    title: "땀나게 운동하기"
  ▶ [[Prototype]]: Object
```

axios()를 사용한 비동기 처리

- axios()를 사용한 비동기 처리 – AxiosPost.jsx

```
const AxiosPost = () => {
    // 할 일 제목을 저장하는 상태
    const [title, setTitle] = useState("");

    // 입력 필드 변경 시 상태 업데이트
    const handleChange = (e) => {
        setTitle(e.target.value);
    }

    // 폼 제출 시 POST 요청을 보내는 함수
    const handleSubmit = (e) => {
        e.preventDefault();
        // JSONPlaceholder API에서 할 일 데이터를 생성합니다.
        axios.post("https://jsonplaceholder.typicode.com/todos", {
            title: title,
            completed: false,
        })
```

axios()를 사용한 비동기 처리

- axios()를 사용한 비동기 처리 – AxiosPost.jsx

```
    .then((response) => {
        console.log("생성된 데이터:", response.data);
        alert("등록 완료!");
        setTitle(""); // 입력 필드 초기화
    })
    .catch((error) => console.error(error));
}

return (
    <div>
      <h2>할 일(To-do) 추가</h2>
      <form onSubmit={handleSubmit}>
        <input
          type="text"
          value={title}
          onChange={handleChange}
          placeholder="할 일을 입력하세요"
        />
        <button type="submit" style={{marginLeft: '6px'}}>등록</button>
      </form>
    </div>
);
```

axios()를 사용한 비동기 처리

- axios()를 사용한 비동기 처리

```
수정된 데이터:                                    AxiosPut.jsx
▼ {id: 1, title: '비타민 먹기', completed: true}  i
    completed: true
    id: 1
    title: "비타민 먹기"
  ▶ [[Prototype]]: Object
```

axios()를 사용한 비동기 처리

- axios()를 사용한 비동기 처리 – AxiosPut.jsx

```
const AxiosPut = () => {
    // 할 일 제목을 저장하는 상태
    const [title, setTitle] = useState("");

    // 입력 필드 변경 시 상태 업데이트
    const handleChange = (e) => {
        setTitle(e.target.value);
    }

    // 폼 제출 시 PUT 요청을 보내는 함수
    const handleSubmit = (e) => {
        e.preventDefault();
        // JSONPlaceholder API에서 할 일 데이터를 업데이트합니다.
        axios.put("https://jsonplaceholder.typicode.com/todos/1", {
            id: 1, // 수정할 데이터의 ID를 명시적으로 포함
            title: title,
            completed: true,
        })
```

axios()를 사용한 비동기 처리

- axios()를 사용한 비동기 처리 – AxiosPut.jsx

```
    .then((response) => {
        console.log("수정된 데이터:", response.data);
        alert("수정 완료!");
        setTitle(""); // 입력 필드 초기화
    })
    .catch((error) => console.error(error));
}

return (
    <div>
      <h2>할 일(To-do) 수정</h2>
      <form onSubmit={handleSubmit}>
        <input
          type="text"
          value={title}
          onChange={handleChange}
          placeholder="할 일을 입력하세요"
        />
        <button type="submit" style={{marginLeft: '6px'}}>수정</button>
      </form>
    </div>
);
```

axios()를 사용한 비동기 처리

- axios()를 사용한 비동기 처리

axios()를 사용한 비동기 처리

- axios()를 사용한 비동기 처리 – AxiosDelete.jsx

```
const AxiosDelete = () => {

    const handleDelete = () => {
        // JSONPlaceholder API에서 할 일 데이터를 삭제합니다.
        axios.delete("https://jsonplaceholder.typicode.com/todos/1")
        .then((response) => {
            console.log("삭제된 데이터:", response.data);
            alert("삭제 완료!");
        })
        .catch((error) => console.error(error));
    }

    return (
        <div>
          <h2>할 일(To-do) 삭제</h2>
            <span>1번 할 일 </span>
            <button onClick={handleDelete}>삭제</button>
        </div>
    );
}
```